

Sesión 1

Fundamentos, Instalación y Primer Despliegue

El nacimiento del agente

Agenda

Bloque	Tema	Tiempo
 1	Presentación, teoría y Setup	40 min
	Descanso	20 min
 2	Hola Mundo en Consola	40 min
	Descanso	20 min
 3	Despliegue en Discord	30 min
 4	Ejemplos prácticos	15 min
	Preguntas	15 min

Objetivo: Pasar de cero a tener un agente conversacional en producción.

Bloque 1

Teoría y Setup

¿Qué es OpenClaw y por qué existe?

¿Qué es un LLM?

Un **Large Language Model** (LLM) es un modelo de lenguaje entrenado con grandes cantidades de texto que puede:

- Responder preguntas
- Generar texto
- Resumir documentos
- Traducir idiomas
- Generar video
- ...

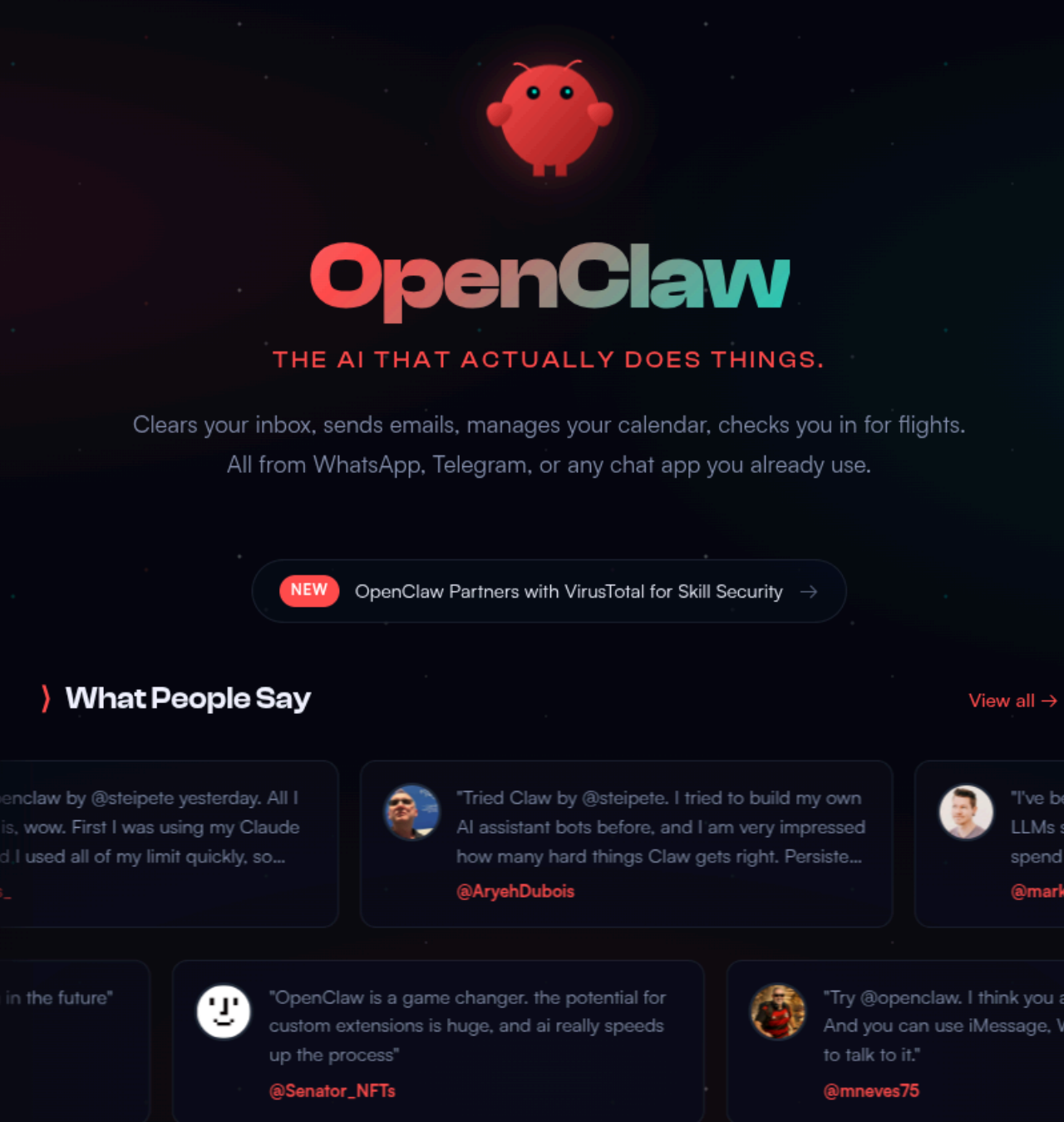
! Pero un LLM solo no puede hacer nada en el mundo real.

No tiene memoria, no puede llamar a APIs, no puede actuar.

LLM vs. Agente Inteligente

	LLM puro	Agente (OpenClaw)
Memoria	✗ Sin memoria entre sesiones	✓ Historial persistente
Acciones	✗ Solo genera texto	✓ Ejecuta Skills/Tools
Integración	✗ API sin estado	✓ Telegram, Slack, Web...
Contexto	✗ Prompt manual	✓ MCP / RAG automático
Orquestación	✗ Un solo modelo	✓ Multi-agente

¿Qué es OpenClaw?



OpenClaw es un framework open source para construir agentes conversacionales que

-  Se conecta a **múltiples canales** (Telegram, web, terminal...)
-  Ejecutan **Skills personalizadas** (APIs, bases de datos, scripts)
-  Mantienen **memoria** de la conversación
-  Admiten **múltiples modelos** (OpenAI, Anthropic, modelos locales)

Peter Steinberger — El Clawdfather

📍 Viena ↔ Londres · @steipete

- 🏢 **2011–2021:** Fundó **PSPDFKit**, el SDK de PDF líder para iOS/Android — bootstrapped hasta millones en ARR, exit en 2021 (\$116M ronda de financiación)
- 📱 **13+ años** desarrollando apps nativas iOS: conocido por *Aspects* (AOP en Objective-C, 10k+ ★) e *InterposeKit*
- 🔬 Contribuidor activo en la comunidad open source; ponente internacional sobre AI y desarrollo ágil
- 🤖 **2024:** Volvió del retiro para "**mess with AI**" — y creó OpenClaw

"Ship beats perfect" — su filosofía de siempre

De PSPDFKit a OpenClaw: el salto a los agentes

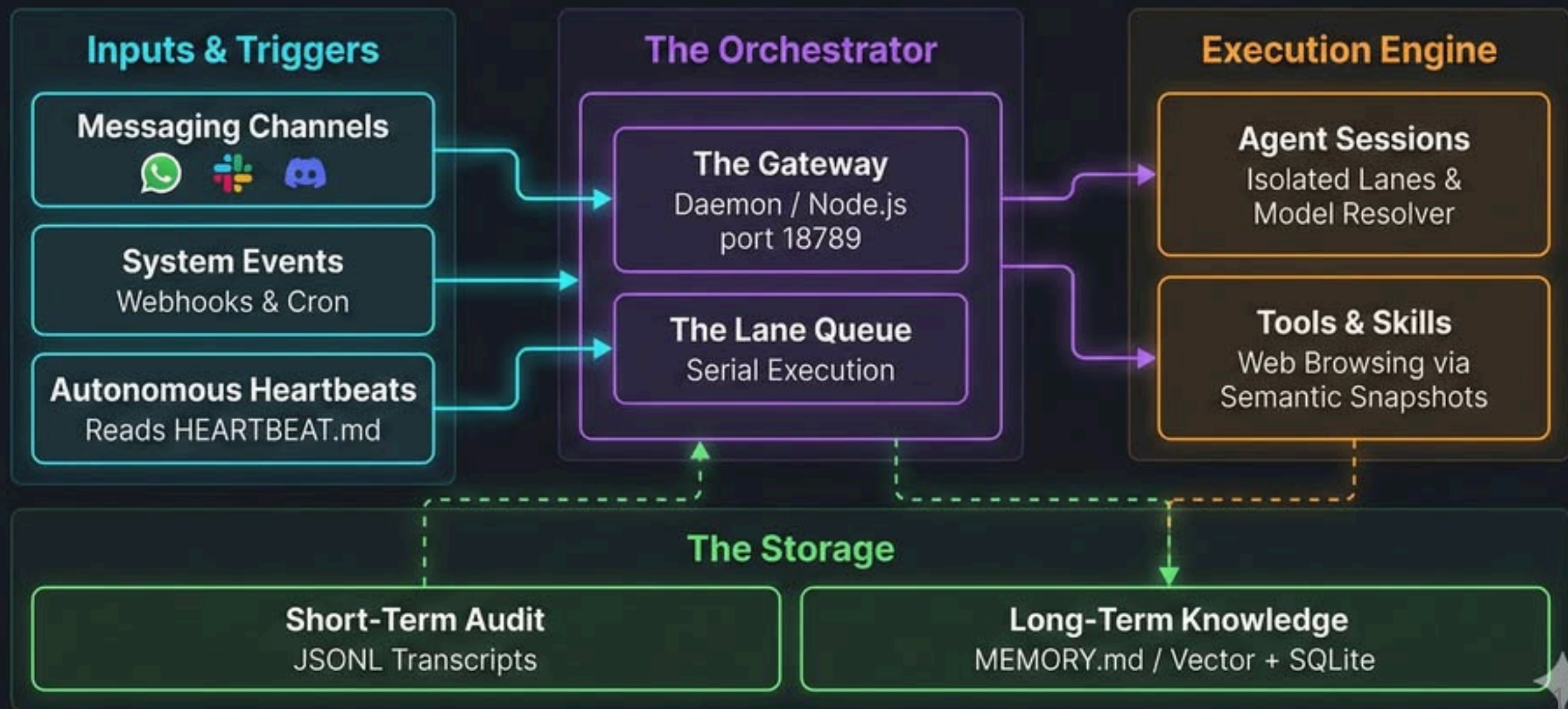
Tras el exit de PSPDFKit, Peter exploró el ecosistema AI y detectó un hueco:

"Todos los LLMs generan texto — nadie construía agentes que realmente actuasen"

De PSPDFKit a OpenClaw: el salto a los agentes

-  Empezó construyendo **herramientas para sí mismo**: CLIs, MCPs, automatizaciones
— más de 50 repos públicos en un año (*Peekaboo, VibeTunnel, Terminator MCP...*)
 -  Nació **OpenClaw**: el agente que *"actually does things"*
— identidad via Markdown, memoria persistente, skills, canales, multi-modelo
 -  Proyecto **open source** desde el primer día, con comunidad de sponsors activa
 -  Divulga su workflow AI en steipete.me y conferencias internacionales
-  *El nombre «Clawdfather» en su GitHub lo dice todo.*

OpenClaw Architecture



Posibilidades de instalación

	VPS	Local + Docker	Local nativo
Seguridad	★★★★☆	★★★★☆	★★☆☆☆
Facilidad	★★☆☆☆	★★★☆☆	★★★★☆
Coste	★★★★★	★★★☆☆	★★★☆☆
Mantenimiento	★★☆☆☆	★★★☆☆	★★☆☆☆
Escalabilidad	★★★★★	★★★☆☆	★★☆☆☆

Posibilidades de instalación

Lectura rápida

- **VPS:** mejor para producción y acceso remoto 24/7
- **Local + Docker:** mejor equilibrio para formación y demos controladas
- **Local nativo:** útil para pruebas rápidas, menos recomendable para equipos

Conceptos básicos

- Modelos
- Sesiones
- Canales

Modelos

¿Qué son los modelos en OpenClaw?

Los **modelos** son los **Large Language Models (LLMs)** que impulsan la inteligencia del agente:

-  **Proveedores principales:** OpenAI (GPT), Anthropic (Claude), Google (Gemini), Meta (Llama)
-  **Modelos locales:** Ollama, LM Studio (para privacidad o sin internet)
-  **Configuración:** Se definen en `openclaw.json` con API keys y parámetros en lugar seguro

Ejemplo básico:

```
"model": {  
  "primary": "openrouter/minimax/minimax-m2.5:free"  
}
```

💡 El modelo es el "cerebro" — determina calidad, velocidad y capacidades del agente.

Por qué elegir un buen modelo

Un buen modelo impacta directamente en la experiencia del usuario:

Aspecto	Importancia
Calidad de respuesta	Comprensión precisa, respuestas coherentes
Velocidad	Tiempo de respuesta (crucial para chat en tiempo real)
Costo	Tokens consumidos vs. presupuesto
Capacidades	Razonamiento, creatividad, manejo de contexto largo
Fiabilidad	Disponibilidad y estabilidad del servicio

Ejemplo comparativo:

- GPT-3.5: Rápido y barato, pero limitado en complejidad
- GPT-4: Más inteligente, pero más caro y lento
- Claude: Excelente en ética y seguridad
- Gemini, Minimax, Qwen...

Múltiples modelos

OpenClaw nos va a permitir configurar múltiples modelos, incluso nos permitirá asignar modelos por canal, sesión, tarea...etc



Es importante configurar siempre, al menos, un modelo de fallback.

La importancia del fallback

¿Por qué un fallback? Porque ningún proveedor es infalible:

-  **Downtime:** OpenAI ha tenido outages (ej: incidentes de 2023-2024)
-  **Límites de rate:** Cuotas diarias/mensuales pueden agotarse
-  **Costos variables:** Cambios en pricing sin aviso
-  **Regulaciones:** Restricciones geográficas o compliance
-  **Diversidad:** Diferentes modelos para diferentes tareas

 Sin fallback, tu agente queda "mudo" si el proveedor principal falla.

Modelos recomendados para empezar

Modelo	Proveedor	Ventajas	Desventajas	Costo aproximado
GPT-4	OpenAI	Inteligente, versátil	Caro, rate limits	\$0.03/1K tokens
Claude 3	Anthropic	Ético, creativo	Menos herramientas	\$0.015/1K tokens
Gemini Pro	Google	Gratuito para pruebas	Menos maduro	\$0.001/1K tokens
Llama 2 7B	Meta (local)	Privado, sin costo	Requiere hardware	\$0 (una vez descargado)

Recomendación inicial:

Mejores prácticas con modelos

-  **Monitorea uso:** Tokens consumidos, latencia, errores
-  **Presupuesto:** Establece límites para evitar sorpresas
-  **Testea:** Prueba prompts en diferentes modelos
-  **Actualiza:** Nuevos modelos salen regularmente
-  **Multi-región:** Usa proveedores con datacenters cercanos

Comando útil:

```
# Ver métricas de uso  
openclaw models status
```

Sesiones

¿Qué es una sesión en OpenClaw?

Una **sesión** es una conversación continua entre un usuario y el agente:

-  **Estado persistente:** El agente recuerda el contexto a lo largo de la interacción
-  **Historial completo:** Todos los mensajes previos se incluyen en cada petición al LLM
-  **Identificador único:** Cada sesión tiene un ID para rastrear y gestionar
-  **Tiempo de vida:** Las sesiones pueden expirar o reiniciarse según configuración

 Sin sesiones, cada mensaje sería independiente — como un LLM básico.

Gestión de sesiones

OpenClaw gestiona las sesiones automáticamente:

Aspecto	Cómo funciona
Creación	Primera interacción inicia una nueva sesión
Persistencia	Historial guardado en base de datos o memoria
Expiración	Configurable (ej: 24h inactivas)
Reinicio	Comando <code>/reiniciar</code> o manual en web
Multi-usuario	Sesiones separadas por canal/usuario

Canales

¿Qué son los canales en OpenClaw?

Un **canal** es la vía por la que los usuarios se comunican con el agente:

-  **Entrada:** Recibe los mensajes del usuario (texto, comandos, eventos)
-  **Procesamiento:** El agente genera la respuesta
-  **Salida:** Devuelve la respuesta por el mismo canal

Canales soportados:

Canal	Tipo	Caso de uso
Terminal (TUI)	Local	Desarrollo y pruebas
Web	HTTP	Panel de administración
Discord	Mensajería	Comunidades, equipos técnicos
Telegram	Mensajería	Bots de atención, demos
Slack	Mensajería	Entornos corporativos
API REST	Programático	Integración en otras apps
...	...	...

Supported channels

- **BlueBubbles** – **Recommended for iMessage**; uses the BlueBubbles macOS server REST API with full feature support (edit, unsend, effects, reactions, group management – edit currently broken on macOS 26 Tahoe).
- **Discord** – Discord Bot API + Gateway; supports servers, channels, and DMs.
- **Feishu** – Feishu/Lark bot via WebSocket (plugin, installed separately).
- **Google Chat** – Google Chat API app via HTTP webhook.
- **iMessage (legacy)** – Legacy macOS integration via imsg CLI (deprecated, use BlueBubbles for new setups).
- **IRC** – Classic IRC servers; channels + DMs with pairing/allowlist controls.
- **LINE** – LINE Messaging API bot (plugin, installed separately).
- **Matrix** – Matrix protocol (plugin, installed separately).
- **Mattermost** – Bot API + WebSocket; channels, groups, DMs (plugin, installed separately).
- **Microsoft Teams** – Bot Framework; enterprise support (plugin, installed separately).
- **Nextcloud Talk** – Self-hosted chat via Nextcloud Talk (plugin, installed separately).
- **Nostr** – Decentralized DMs via NIP-04 (plugin, installed separately).

- Integraciones

A día de hoy: BlueBubbles, Discord, Feishu, Google Chat, iMessage (legacy), IRC, LINE, Matrix, Mattermost, Microsoft Teams, Nextcloud Talk, Nostr, Signal, Slack, Synology Chat, Telegram, Tlon, Twitch, Voice Call, WebChat, WhatsApp, Zalo, Zalo Personal.

¿Por qué son importantes los canales?

Los canales son la **interfaz entre el agente y el mundo real**:

- 🌐 **Alcance**: El mismo agente puede estar en Discord, Telegram y la web simultáneamente
- 🎯 **Contexto**: Cada canal puede tener su propia identidad o comportamiento
- 🔒 **Control**: Puedes restringir qué usuarios o canales tienen acceso
- 📊 **Trazabilidad**: Cada sesión queda asociada al canal de origen

💡 Sin canales, el agente solo existe en local — los canales son los que lo llevan a producción.

Cómo funciona un canal

El flujo de un mensaje en cualquier canal es siempre el mismo:

```
Usuario escribe mensaje
↓
Canal lo recibe
↓
OpenClaw crea/recupera sesión
↓
Carga workspace (SOUL, IDENTITY, TOOLS...)
↓
LLM genera respuesta
↓
Canal devuelve la respuesta
↓
Usuario la recibe
```

Clave: El canal es transparente — el agente no sabe si habla por Discord o Telegram.

Configuración de canales

Los canales se configuran en `openclaw.json` :

```
"channels": {  
  "discord": {  
    "enabled": true,  
    "token": "${DISCORD_BOT_TOKEN}",  
    "guildId": "${DISCORD_GUILD_ID}"  
  },  
  "telegram": {  
    "enabled": true,  
    "token": "${TELEGRAM_BOT_TOKEN}"  
  },  
  ...  
}
```

💡 **Puedes tener múltiples canales activos a la vez** — cada uno con su propia configuración de acceso.

Local + docker

- VirtualBox

Variables de entorno

Crear el fichero `.env` en la raíz del proyecto

Importante: Debes establecer el TimeZone (OPENCLAW_TZ) adecuado para evitar problemas con la aplicación web

💡 **Nunca subas el `.env` a Git.** Está en el `.gitignore` por defecto.

Go live!

Primeros pasos

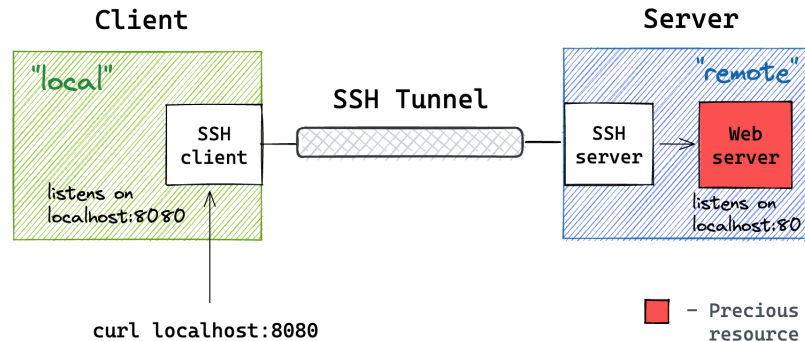
Acceso básico

- Arquitectura final (MI PC > Virtual Box > Docker > OpenClaw)
- Acceso a los contenedores
- Acceso a TUI (*Terminal User Interface*)

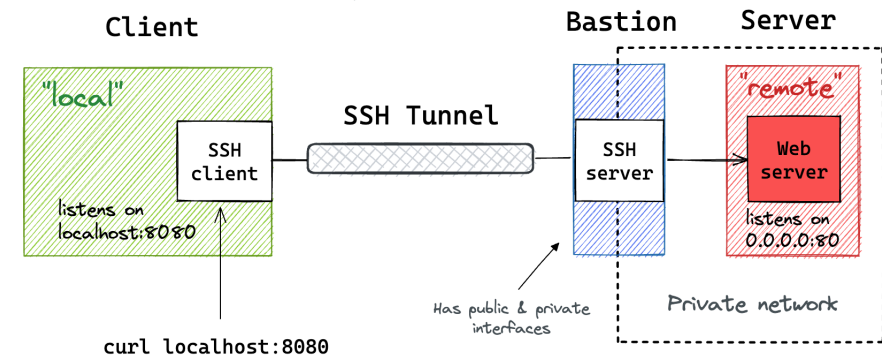
Acceso básico

SSH Tunneling: <https://iximiuz.com/en/posts/ssh-tunnels/>

Short form → "local" address "remote" address sshd address
`ssh -L 8080 :localhost:80 user@server`
Long form → `ssh -L localhost:8080:localhost:80 user@server`
local address tells ssh client where to start listening
remote address tells sshd server where to forward traffic to



Short form → "local" address "remote" address sshd address
`ssh -L 8080 :server:80 user@bastion`
Long form → `ssh -L localhost:8080:server:80 user@bastion`
local address tells ssh client where to start listening
remote address tells sshd server where to forward traffic to



"remote" address "local" address sshd address
`ssh -R 0.0.0.0:8080:localhost:80 user@gateway`
remote address tells sshd server where to start listening
local address tells ssh client where to forward traffic to

Client

Gateway

"remote" address "local" address sshd address
`ssh -R 0.0.0.0:8080:server:80 user@gateway`
remote address tells sshd server where to start listening
local address tells ssh client where to forward traffic to

Acceso básicos

Comandos habituales:

- Lanzar comandos: `docker compose run -it openclaw-cli <COMANDO>`
- Conectar con la máquina: `docker compose exec -it openclaw-gateway bash`
- Configurar openclaw `openclaw configure`
- Abrir TUI `openclaw tui`

TUI — Terminal User Interface

La **TUI** es la interfaz de texto interactiva de OpenClaw, accesible desde el terminal:

```
openclaw tui
```

-  **Sin necesidad de navegador** — todo desde la terminal
-  **Chat directo** con el agente en tiempo real
-  **Gestión de sesiones** — ver historial, cambiar agente activo
-  **Acceso al workspace** — editar `SOUL.md` , `IDENTITY.md` ... sin salir
-  **Diagnóstico rápido** — ver logs, estado del modelo, errores

 La TUI es la herramienta ideal para hacer la primera configuración del agente.

TUI — Primera conversación

Al abrir la TUI por primera vez, el agente usa los valores por defecto del workspace:

1. **Abre la TUI:** `openclaw tui`
2. **Escribe un mensaje** — el agente responde con su identidad actual
3. **Edita** `SOUL.md` para darle instrucciones base (quién es, cómo responde)
4. **Edita** `IDENTITY.md` para definir nombre, empresa, tono
5. **Reinicia la sesión** — los cambios se aplican en la siguiente conversación

TUI — Primera conversación

Pantalla de presentación

```
OpenClaw TUI
Sesión: abc123 | Modelo: minimax-m2.5

Tú: Hola, ¿quién eres?
Agente: Soy tu asistente. ¿En qué te ayudo?

> _
```

Access

dashboard connects and how it authenticates.

et URL

7.0.0.1:18789

(not stored)

h or shared password

ct

Refresh

Click Connect to apply connection changes.

Gateway Token

CLAWDBOT_GATEWAY_TOKE

Default Session Key

agent:main:main

Snapshot

Latest gateway handsh

STATUS

Connected

TICK INTERVAL

n/a

Use Channels to link W
iMessage.

beacons in the last 5 minutes.

SESSIONS

1

Recent session keys tracked by the gateway.

Web

- Nos permite configurar lo mismo que por configure
- Acceso simplificado y seguro
- Permite un "acceso totalmente remoto"

Nuestro Primer canal



Hola Mundo en Consola

```
Hola
```

```
¡Hola! 🖐️
```

```
Acabo de despertar – workspace limpio, sin memoria todavía. Vosotros sabéis quién soy, pero yo todavía no lo sé.
```

```
Según el protocolo de inicio, tendríamos que figuring out:
```

1. **Cómo me llamo** – ¿qué nombre me ponéis?
2. **Qué soy** – asistente de IA está bien, pero igual Preferís algo más raro
3. **Mi vibe** – formal, casual, enrollado, directo...
4. **Y vosotras** – quién eres, cómo queréis que os llame, zona horaria

```
¿Empezamos? O simplemente querías saludar y ya continuamos luego – sin presión. 😊
```

El archivo de configuración

OpenClaw se configura con un fichero `agent.json`, pero también podemos editar la configuración mediante:

- `openclaw configure`
- Web

El System Prompt: el alma del agente

El **System Prompt** es la instrucción base que define la *identidad* del agente:

```
Eres el recepcionista de TechCorp.  
Tu nombre es Alex.  
Responde siempre en español.  
Sé conciso: máximo 3 frases por respuesta.
```

Información que conoces:

- Horario: Lunes a Viernes, 9:00 - 18:00
- Teléfono de soporte: 900 123 456
- Email: hola@techcorp.es

 Un buen System Prompt hace el 80% del trabajo.
Define **quién es, qué sabe y cómo debe responder.**

Anatomía de un buen System Prompt

```
[ROL] Eres X, trabajas para Y.  
[COMPORTAMIENTO] Eres amable / técnico / formal...  
[RESTRICCIONES] No hagas A, no compartas B.  
[CONOCIMIENTO] Información relevante del dominio.  
[FORMATO] Responde en bullets / máx N palabras / en idioma X.
```

Ejemplo de restricciones útiles:

- "Si te preguntan por precios, deriva al equipo comercial."
- "No inventes información. Si no sabes, dilo."
- "No respondas sobre temas que no sean de TechCorp."

La memoria de la conversación

OpenClaw mantiene el **historial** de la sesión automáticamente:

Tú: Me llamo Carlos.

Alex: ¡Encantado, Carlos! ¿En qué puedo ayudarte?

Tú: ¿Recuerdas cómo me llamo?

Alex: Claro, te llamas Carlos. ¿Hay algo más en lo que pueda ayudarte?

Esto es posible porque cada mensaje incluye el historial previo al LLM:

[system] Eres el recepcionista de TechCorp...

[user] Me llamo Carlos.

[assistant] ¡Encantado, Carlos!...

[user] ¿Recuerdas cómo me llamo? ← el LLM "ve" todo lo anterior

 **Nuestro segundo canal: Discord**

Paso 1: Crear la aplicación en Discord Developer Portal

1. Ve a discord.com/developers/applications
2. Clic en **New Application** → ponle un nombre (ej: TechCorp Bot)
3. Ve a la sección **Bot** → clic en **Add Bot**
4. En **Token**, clic en **Reset Token** y cópialo

--

Paso 2: Paso 2: Obtener el Guild ID (servidor)

1. Activa el **Modo Desarrollador** en Discord (Ajustes → Avanzado)
2. Click derecho sobre tu servidor → **Copiar ID del servidor**

Paso 3: Configurar OpenClaw

En `.env`:

```
DISCORD_BOT_TOKEN=tu_token_aquí  
DISCORD_GUILD_ID=tu_guild_id_aquí
```

A partir de aquí, utilizamos `openclaw configure`

Paso 4: Invitar el bot al servidor

1. En el portal, ve a **OAuth2** → **URL Generator**
2. Scopes: `bot` · Permissions: `Send Messages`, `Read Message History`
3. Copia la URL generada y ábrela para invitar el bot

Paso 5: Arrancar y probar

- Inicia OpenClaw y escribe en el canal del servidor
- Comparte el servidor con un compañero para que lo pruebe

Bonus: Cambia `soul.md` para que el bot se presente con tu empresa ficticia.

Ejercicios adicionales



Ejercicio 3 — La lista de la compra

Objetivo: Que el agente gestione una lista de la compra persistente, modificable desde cualquier canal.

Cómo funciona:

- El agente guarda la lista en `workspace/lista_compra.md`
- La lee al arrancar cada conversación
- Puede añadir, tachar y listar ítems mediante lenguaje natural

Pasos:

1. Edita `SOU.L.md` para indicar al agente que gestiona una lista de la compra (opcional)
2. Dile: *"Añade leche, pan y huevos a la lista"*
3. Dile: *"Ya compré el pan, márcalo"*
4. Dile: *"¿Qué me falta comprar?"*
5. Prueba desde **Discord** y comprueba que la lista persiste entre canales

Ejercicio 3 — La lista de la compra (cont.)

Ejemplo de `SOUL.md` :

Eres un asistente doméstico que gestiona la lista de la compra.
La lista se guarda en `workspace/lista_compra.md`.
Cuando el usuario añada algo, actualiza el fichero con - [] ítem.
Cuando diga que ya lo compró, cámbialo a - [x] ítem.
Al preguntar qué falta, muestra solo los ítems sin tachar.

Ejemplo de conversación:

Tú: Añade leche y café

Agente:  Añadidos. Tu lista ahora tiene: leche, café, pan...

Tú: Ya compré el café

Agente:  Marcado. Te queda: leche, pan...

Tú: ¿Qué me falta?

Agente:  Pendiente: leche, pan

 Abre `workspace/lista_compra.md` y verifica que el fichero se ha actualizado.



Ejercicio 4 — Resumen de documentos

Objetivo: Que el agente lea documentos de alumnos desde el workspace, genere un resumen por alumno y cree un informe consolidado con citas guardado en disco.

Cómo funciona:

- Los documentos están en `workspace/alumnos/` (uno por alumno)
- El agente los lee al arrancar la conversación
- Genera resúmenes individuales y un informe final con citas

Pasos:

1. Copia los ficheros de alumnos en `workspace/alumnos/` (ya están en `session01/data/`)
2. Actualiza `SOUL.md` para indicar al agente que es un evaluador académico
3. Pide: *"Haz un resumen del trabajo de cada alumno"*
4. Pide: *"Genera un informe consolidado con citas y guárdalo en `workspace/informe_final.md`"*
5. Abre `workspace/informe_final.md` y verifica el resultado

Ejercicio 4 — Resumen de documentos (cont.)

Ejemplo de `SOUL.md` :

Eres un evaluador académico. Tienes acceso a los trabajos de los alumnos en `workspace/alumnos/`. Al resumir, sé objetivo y conciso. Usa el formato: `## Alumno X · [título del trabajo]`
Cuando generes el informe final, incluye citas textuales entre comillas y guárdalo en `workspace/informe_final.md`.

Ejemplo de conversación:

Tú: Resume el trabajo de cada alumno

Agente: ## Alumno 01 · Redes Neuronales
Analiza arquitecturas CNN aplicadas a visión...

Alumno 02 · NLP con Transformers
Compara BERT y GPT en tareas de clasificación...

Tú: Genera el informe consolidado y guárdalo

Agente:  Informe guardado en workspace/informe_final.md
Incluye resúmenes y citas de 5 trabajos.

 Comprueba que `workspace/informe_final.md` existe y contiene las citas correctas.

Ejercicio 5 — Resumen de noticias por cron

Objetivo: Programar una tarea diaria que busque noticias en `marca.es`, las resuma y las envíe automáticamente por **Discord**.

Cómo funciona:

- Se registra un **cron job** con `openclaw cron add` apuntando al canal Discord
- El agente visita `marca.es`, extrae los titulares y genera un resumen
- Lo envía **directamente a Discord** sin intervención manual

Cron vs Heartbeat

 docs.openclaw.ai/automation/cron-vs-heartbeat

	Heartbeat	Cron (isolated)
Timing	Cada N minutos (aprox.)	Expresión cron exacta
Contexto	Sesión principal (memoria)	Sesión limpia por ejecución
Uso ideal	Monitoreo, inbox, calendario	Informes diarios, tareas precisas
Coste	Menor (una llamada batcheada)	Una llamada por job
Historial	Compartido con el agente	No contamina la sesión principal
Entrega	Solo si hay algo importante	- - announce → directo al canal

¿Cuándo usar cada uno?

-  **Heartbeat** → *"Vigila mi email cada 30 min y avísame si hay algo urgente"*
 -  **Cron** → *"Mándame el resumen de noticias todos los días a las 8:00"*
-  Lo más eficiente es **combinar ambos**: heartbeat para monitoreo continuo + cron para tareas puntuales con horario fijo.

Pasos:

1. Asegúrate de que el canal Discord está activo y configurado
2. Registra el cron desde la **TUI o terminal** con `openc law cron add`
3. Edita `SOUL.md` para que el agente sepa resumir noticias deportivas
4. **Verifica en la web** (`openc law web`) que el job aparece activo en el panel de cron
5. Espera la primera ejecución (o fuerza una manual) y comprueba el mensaje en Discord



Ejercicio 5 — Resumen de noticias (cont.)

Registrar el cron job:

```
openclaw cron add \  
  --name "Resumen deportivo" \  
  --cron "0 8 * * *" \  
  --tz "Europe/Madrid" \  
  --session isolated \  
  --message "Busca los 5 titulares más importantes de https://www.marca.es hoy. Incluye titular, párrafo de contexto y enlace. Formato: 📝 **Resumen deportivo – [fecha]**" \  
  --announce \  
  --channel discord
```

Verificación en el panel web:

openclaw web → Sección "Cron Jobs"

☒ Resumen deportivo	0 8 * * *	discord	 Run	
---	-----------	---------	---	--

Ejemplo de mensaje recibido en Discord:

 Resumen deportivo – 25 mar 2026

1 Real Madrid golea al Barça en el Clásico (3-0)

 marca.es/futbol/primera-division/...

2 Alcaraz confirma su presencia en Roland Garros

 marca.es/tenis/...

...



Cambia la URL por cualquier otra fuente y ajusta `--tz` a tu zona horaria.

Resumen de la sesión

- ✓ Entendemos la diferencia entre LLM puro y agente inteligente
- ✓ Sabemos instalar y configurar OpenClaw
- ✓ Creamos un System Prompt efectivo con identidad y restricciones
- ✓ El agente mantiene memoria de la conversación
- ✓ El bot está desplegado y funcional en Telegram

Próxima sesión: Skills

"El agente ya sabe hablar... ahora le damos manos."

En la próxima sesión aprenderemos a crear **Skills** (herramientas) para que el agente:

-  Consulte APIs en tiempo real (clima, precios, datos...)
-  Registre leads en una hoja de cálculo
-  Ejecute acciones en el mundo real de forma autónoma

Referencias y recursos

- 📖 Documentación oficial de OpenClaw: `github.com/tu-org/openclaw`
- 🤖 API de Telegram Bots: `core.telegram.org/bots/api`
- 🔑 OpenAI API Keys: `platform.openai.com/api-keys`
- 🔑 Anthropic API Keys: `console.anthropic.com`
- 🦌 Ollama (modelos locales): `ollama.ai`
- 📐 Guía de System Prompts: *"The Art of the System Prompt"* — Anthropic Blog